DOKUMEN PANDUAN TEKNIS & NASKAH PRESENTASI VIDEO — UJIAN CHUNIN CLOUD COMPUTING

Studi Kasus 6: Papan Catatan Tugas Kelompok (Team Task & Note List)

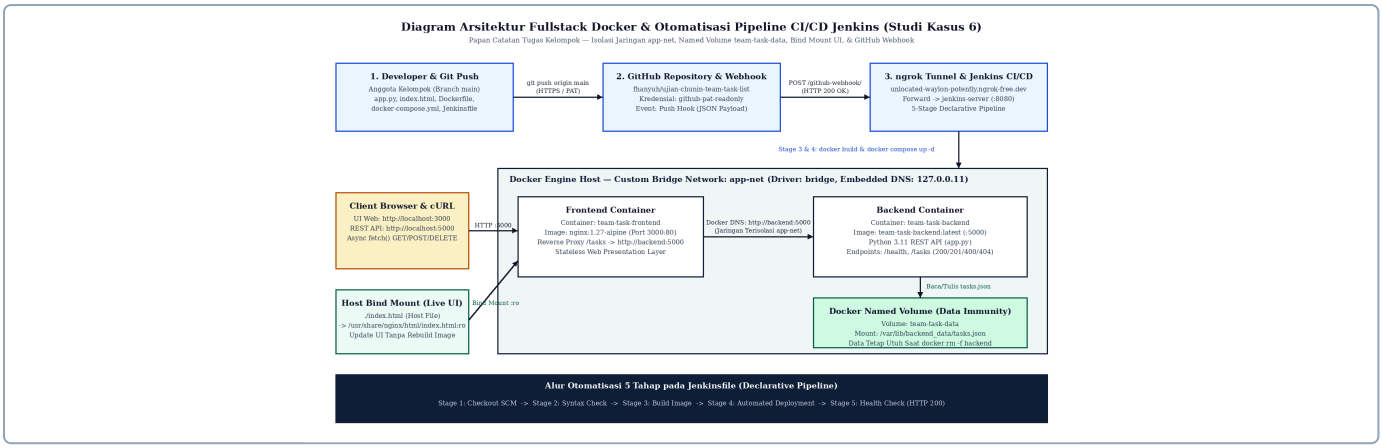
Orkestrasi Fullstack Docker Compose, Isolasi Custom Bridge Network (`app-net`), Persistensi Named Volume (`team-task-data`), Live Bind Mount UI, serta Otomatisasi Pipeline CI/CD Jenkins & GitHub Webhook

No	Nama Lengkap Anggota	NIM	Pembagian Peran Teknis & Segmen Presentasi Video
1	**Rafhan Mazaya Fathurrahman**	`101012300238`	**Segmen 1 (00:00 – 01:45)**: Pembukaan, Arsitektur `docker-compose.yml` , Custom Bridge Network `app-net` , & Komunikasi DNS Antar-Kontainer
2	**Zahra Talitha Putri**	`101012300192`	**Segmen 2 (01:45 – 03:30)**: Kode Backend Python REST API (`app.py`), `Dockerfile` , & Demo Live CRUD Asynchronous `fetch()` (`GET` , `POST` , `DELETE`)
3	**Clara Novita Fitriani**	`101012330008`	**Segmen 3 (03:30 – 05:15)**: Pembuktian *Data Immunity* Named Volume (`docker rm -f team-task-backend`) & Live Bind Mount Frontend (`index.html`)
4	**Rafif Riqullah Siregar**	`101012300320`	**Segmen 4 (05:15 – 07:15)**: Penjelasan 5-Stage `Jenkinsfile` , Integrasi Kredensial GitHub PAT & Webhook via `ngrok` , serta Demo `git push` Otomatis

Repositori GitHub Publik: `https://github.com/fhanyuh/ujian-chunin-team-task-list` | **Frontend Web UI:** `http://localhost:3000` | **Backend API:** `http://localhost:5000` | **Jenkins CI/CD:** `http://localhost:8080/job/ujian-chunin-pipeline/`

Bagian 1 — Arsitektur Teknis & Spesifikasi Sistem Fullstack

Sistem **Papan Catatan Tugas Kelompok (Studi Kasus 6)** dirancang mengikuti arsitektur mikroservis terisolasi di atas Docker Engine. Layanan terbagi menjadi dua kontainer utama yang berkomunikasi melalui jaringan internal `app-net` dan diotomatisasi sepenuhnya oleh `jenkins-server` melalui GitHub Webhook.



Gambar 1: Diagram Arsitektur Fullstack Docker Compose, Custom Bridge Network app-net, Named Volume team-task-data, dan CI/CD Jenkins.

Komponen	Nama Kontainer / Objek	Spesifikasi Teknis & Lokasi Mount	Fungsi Utama
Backend Service	`team-task-backend`	Image `team-task-backend:latest` (`python:3.11-slim`), Port `5000:5000`	Menyediakan REST API CRUD (`GET /tasks` , `POST /tasks` , `PATCH /tasks?index=X` , `DELETE /tasks?index=X`) dan validasi HTTP 200/201/400/404.
Named Volume	`team-task-data`	Dipasang ke `/var/lib/backend_data/tasks.json` di kontainer backend	Menjamin **Data Immunity** sehingga seluruh data tugas kelompok tetap aman meskipun kontainer backend dihapus paksa (`docker rm -f`).
Frontend Service	`team-task-frontend`	Image `nginx:1.27-alpine` , Port `3000:80` , Reverse Proxy ke `http://backend:5000`	Menyajikan antarmuka web modern dan meneruskan panggilan asynchronous `fetch()` ke kontainer backend melalui Docker DNS.
Host Bind Mount	`./index.html`	Dipasang ke `/usr/share/nginx/html/index.html:ro`	Memungkinkan pembaruan antarmuka pengguna (HTML/CSS/JS) tampil seketika di browser tanpa perlu melakukan *rebuild* image kontainer.
Custom Network	`app-net`	Driver `bridge` dengan Docker Embedded DNS (`127.0.0.11`)	Mengisolasi lalu lintas jaringan internal dan mengizinkan resolusi nama layanan `backend` secara otomatis tanpa alamat IP statis.

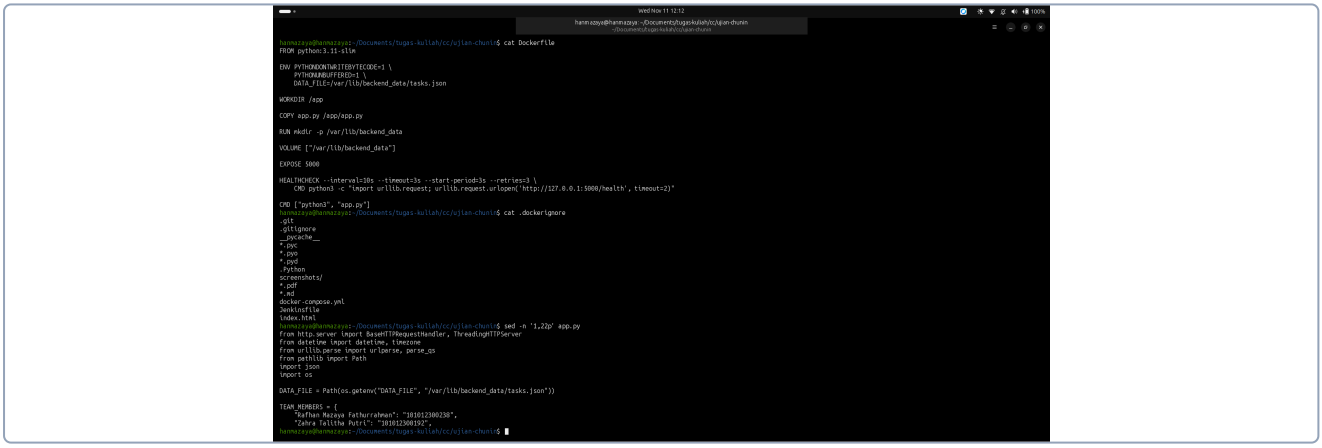
Komponen	Nama Kontainer / Objek	Spesifikasi Teknis & Lokasi Mount	Fungsi Utama
CI/CD Pipeline	`ujian-chunin-pipeline`	Declarative `Jenkinsfile` (5 Stage) + GitHub PAT + `ngrok` Webhook	Membangun ulang image, menyinkronkan Bind Mount, menjalankan `docker compose up -d`, dan menguji kesehatan layanan otomatis saat `git push`.

Bagian 2 — Tutorial Penggunaan End-to-End (Lengkap dengan Bukti Eksekusi)

Langkah 1: Verifikasi Kode Sumber Backend (app.py) & Dockerfile

Pastikan berkas `Dockerfile` menyalin `app.py` ke dalam kontainer berbasis `python:3.11-slim` dan menetapkan variabel lingkungan `DATA_FILE=/var/lib/backend_data/tasks.json` yang terhubung ke volume persisten.

```
cd ~/Documents/tugas-kuliah/cc/ujian-chunin
cat Dockerfile
cat .dockerignore
sed -n '1,22p' app.py
```

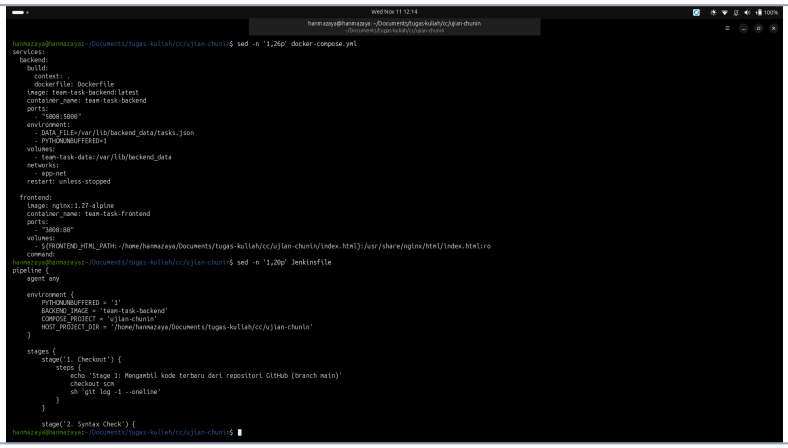


Gambar 2: Pemeriksaan berkas Dockerfile, .dockerignore, dan konfigurasi penyimpanan JSON pada app.py di terminal.

Langkah 2: Verifikasi Orkestrasi docker-compose.yml & Jenkinsfile

Periksa konfigurasi `docker-compose.yml` yang mendefinisikan layanan `backend`, `frontend`, Named Volume `team-task-data`, dan Custom Bridge Network `app-net`, serta berkas `Jenkinsfile` 5 tahap.

```
sed -n '1,26p' docker-compose.yml
sed -n '1,20p' Jenkinsfile
```

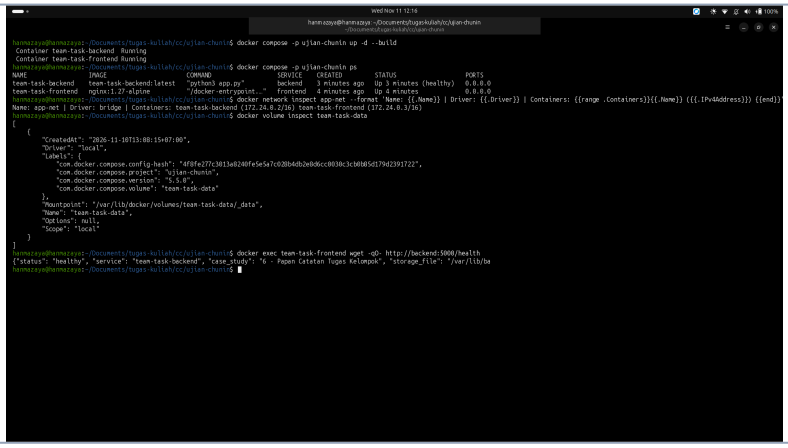


Gambar 3: Konfigurasi layanan pada docker-compose.yml dan deklarasi stage pada Jenkinsfile.

Langkah 3: Menjalankan Stack Docker Compose & Verifikasi Isolasi Jaringan / Volume

Jalankan seluruh layanan dengan `docker compose -p ujian-chunin up -d --build`, lalu buktikan bahwa kedua kontainer berada di jaringan `app-net`, volume `team-task-data` aktif, dan kontainer frontend dapat memanggil `http://backend:5000/health` melalui DNS Docker.

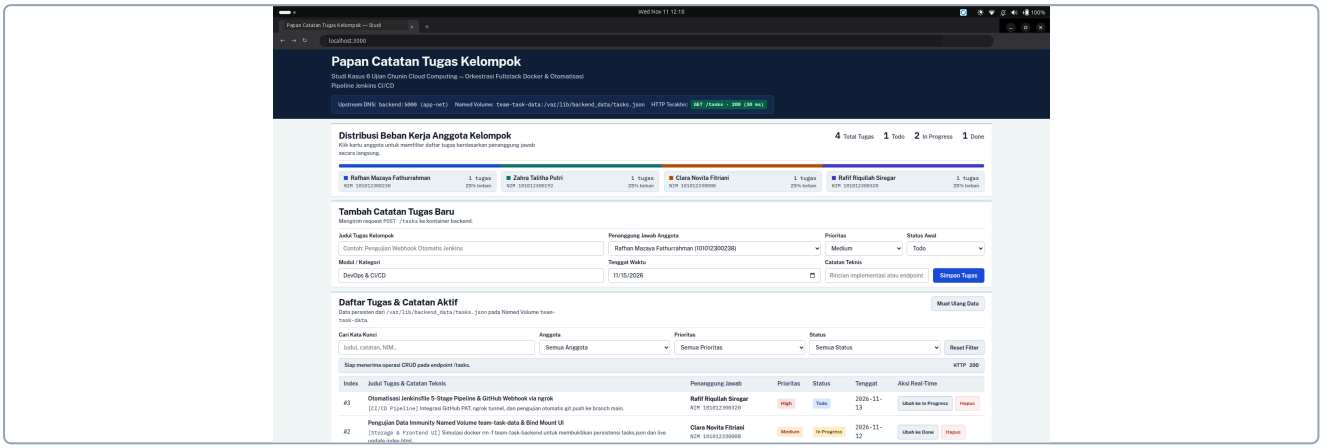
```
docker compose -p ujian-chunin up -d --build
docker compose -p ujian-chunin ps
docker network inspect app-net --format 'Name: {{.Name}} | Driver: {{.Driver}} | Containers:
{{range .Containers}}{{.Name}} ({{.IPv4Address}}) {{end}}'
docker volume inspect team-task-data
docker exec team-task-frontend wget -qO- http://backend:5000/health
```



Gambar 4: Bukti eksekusi docker compose up, inspeksi jaringan app-net, volume team-task-data, dan resolusi DNS internal backend:5000.

Langkah 4: Mengakses Antarmuka Web Papan Catatan Tugas Kelompok (GET /tasks)

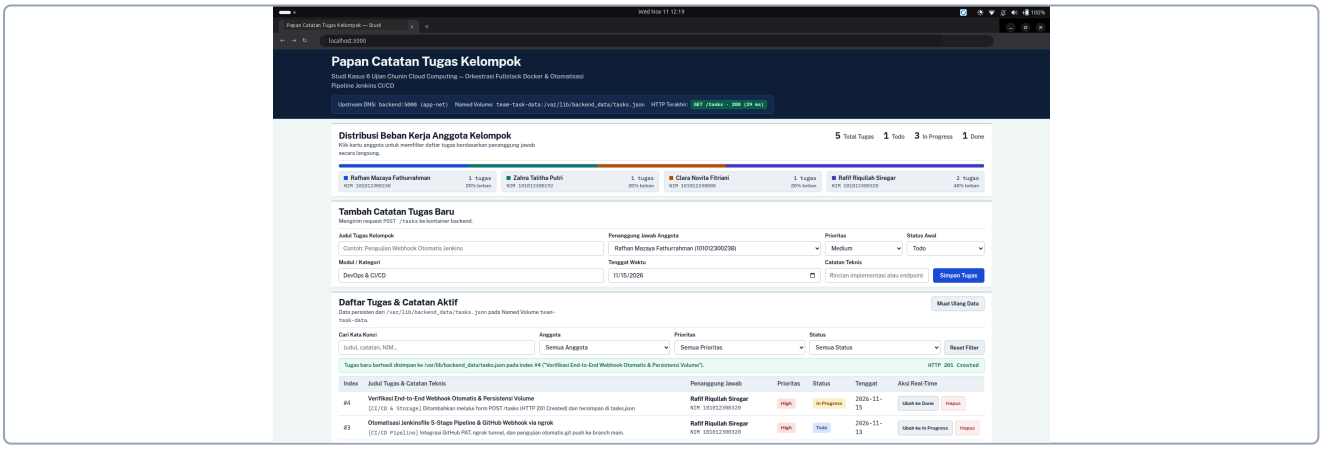
Buka peramban pada `http://localhost:3000`. Antarmuka otomatis mengambil daftar tugas dari backend menggunakan `fetch('/tasks')`, menampilkan matriks distribusi beban kerja 4 anggota kelompok, strip telemetri HTTP, serta tabel tugas aktif.



Gambar 5: Tampilan awal Papan Catatan Tugas Kelompok pada http://localhost:3000 dengan distribusi tugas 4 anggota kelompok.

Langkah 5: Menambahkan Catatan Tugas Baru Melalui UI (POST /tasks — 201 Created)

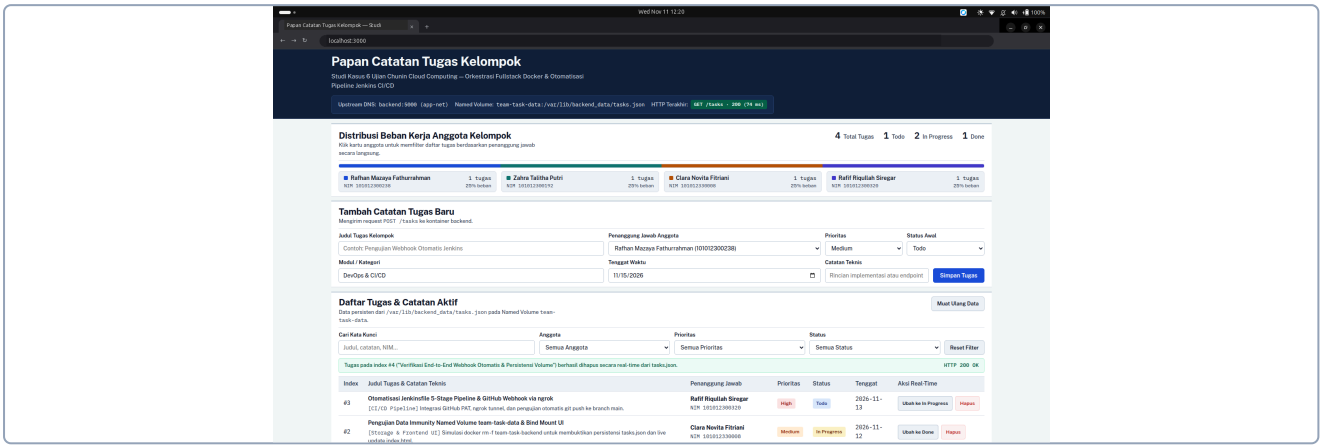
Isi formulir **Tambah Catatan Tugas Baru** (Judul, Penanggung Jawab, Prioritas, Status, Kategori, Tenggat, dan Catatan Teknis) lalu klik **Simpan Tugas**. Permintaan `POST /tasks` dikirim secara asynchronous dan memperbarui tabel serta proporsi beban kerja tanpa me-refresh halaman.



Gambar 6: Tugas baru berhasil ditambahkan pada index #4 (HTTP 201 Created) dan langsung muncul di baris teratas tabel.

Langkah 6: Menghapus / Menyelesaikan Tugas Secara Real-Time (DELETE /tasks?index=X)

Klik tombol **Hapus** pada salah satu baris tugas. Frontend mengirimkan request `DELETE /tasks?index=4` ke backend, menghapus entri dari `tasks.json`, dan langsung memperbarui tabel beserta metrik tugas.

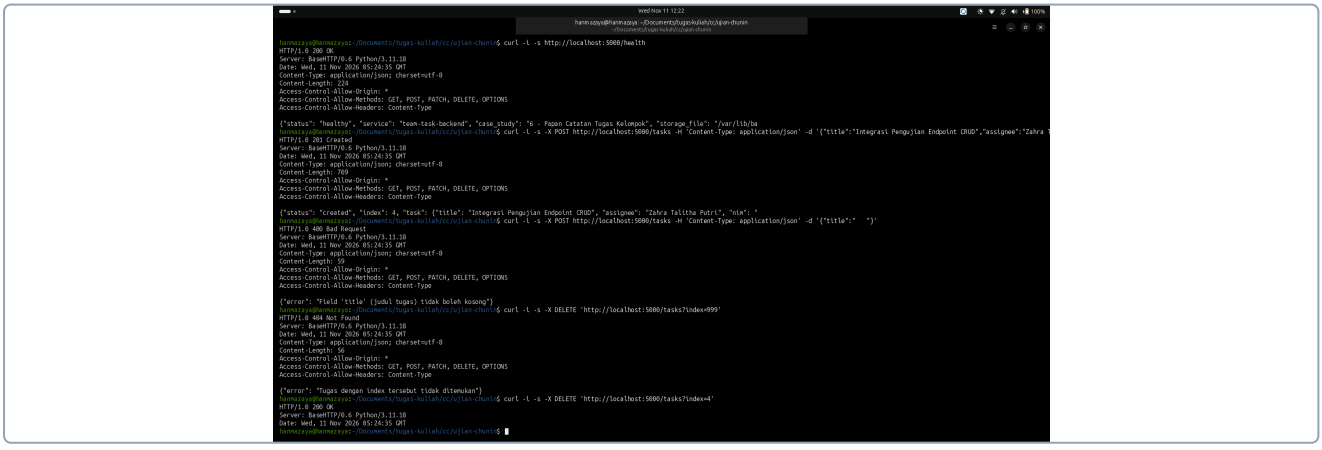


Gambar 7: Penghapusan tugas pada index #4 secara real-time menghasilkan respons HTTP 200 OK dan memperbarui tabel seketika.

Langkah 7: Pengujian Matriks Endpoint REST API & Validasi Error via cURL

Uji ketahanan validasi backend menggunakan `curl` untuk memastikan setiap skenario mengembalikan kode status HTTP yang tepat: `200 OK`, `201 Created`, `400 Bad Request` (input kosong), dan `404 Not Found` (index tidak ditemukan).

```
curl -i -s http://localhost:5000/health
curl -i -s -X POST http://localhost:5000/tasks -H 'Content-Type: application/json' -d
'{"title":"Integrasi Pengujian Endpoint CRUD","assignee":"Zahra Talitha Putri","priority":"High"}'
curl -i -s -X POST http://localhost:5000/tasks -H 'Content-Type: application/json' -d '{"title":"
"}'
curl -i -s -X DELETE 'http://localhost:5000/tasks?index=999'
curl -i -s -X DELETE 'http://localhost:5000/tasks?index=4'
```

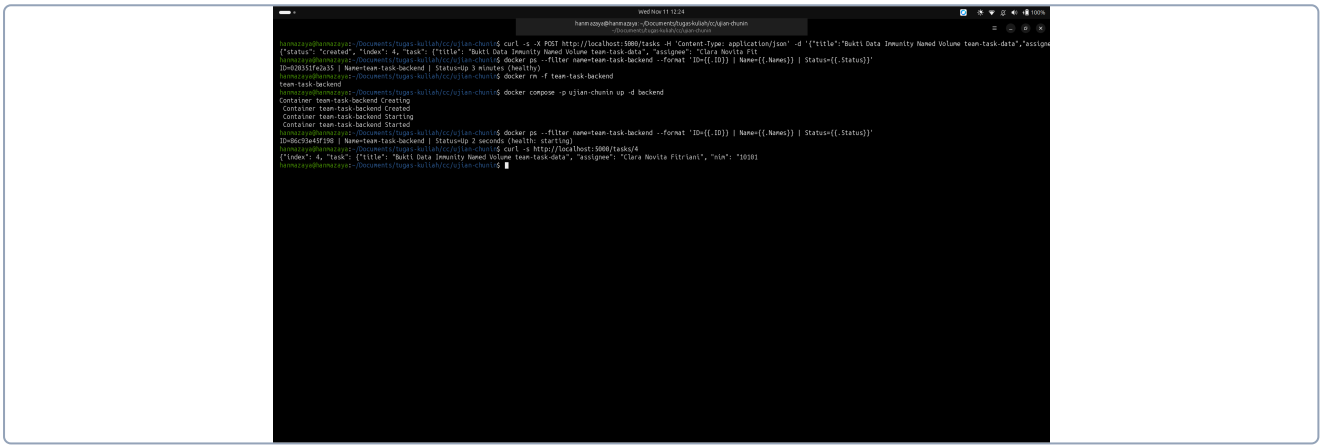


Gambar 8: Bukti pengujian endpoint REST API pada terminal yang membuktikan respons HTTP 200, 201, 400, dan 404.

Langkah 8: Simulasi Data Immunity pada Named Volume (docker rm -f team-task-backend)

Simpan tugas baru ke backend, catat Container ID aktif, lalu hancurkan kontainer backend secara paksa dengan `docker rm -f team-task-backend`. Nyalakan kembali kontainer backend baru dengan `docker compose -p ujian-chunin up -d backend` dan periksa bahwa data pada index ke-4 tetap utuh di dalam volume `team-task-data`.

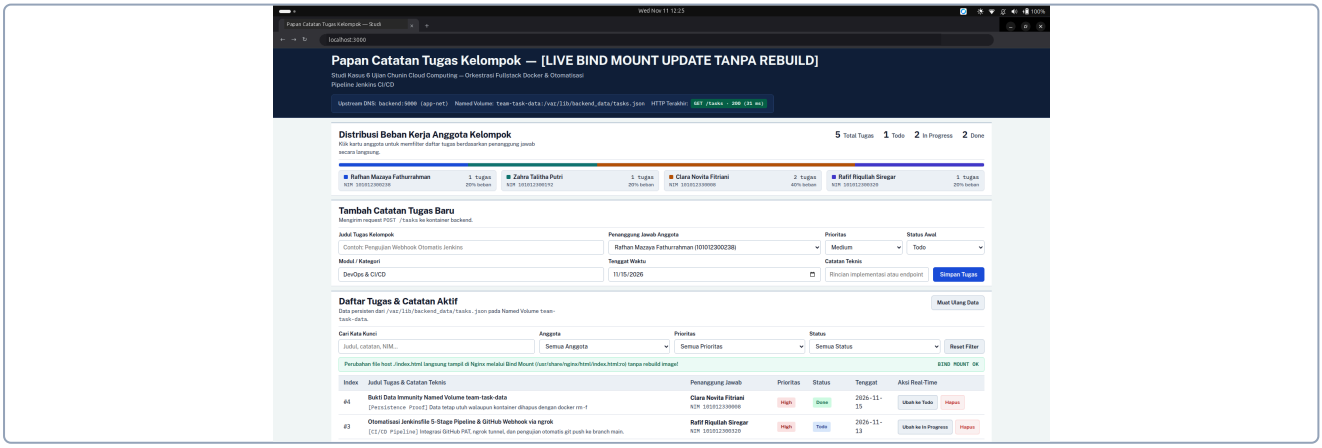
```
curl -s -X POST http://localhost:5000/tasks -H 'Content-Type: application/json' -d '{"title":"Bukti
Data Immunity Named Volume team-task-data","assignee":"Clara Novita
Fitriani","priority":"High","status":"Done"}'
docker rm -f team-task-backend
docker compose -p ujian-chunin up -d backend
curl -s http://localhost:5000/tasks/4
```



Gambar 9: Bukti Data Immunity — meskipun Container ID berganti setelah docker rm -f, data tugas di tasks.json tetap utuh.

Langkah 9: Simulasi Live Bind Mount pada Kontainer Frontend Tanpa Rebuild Image

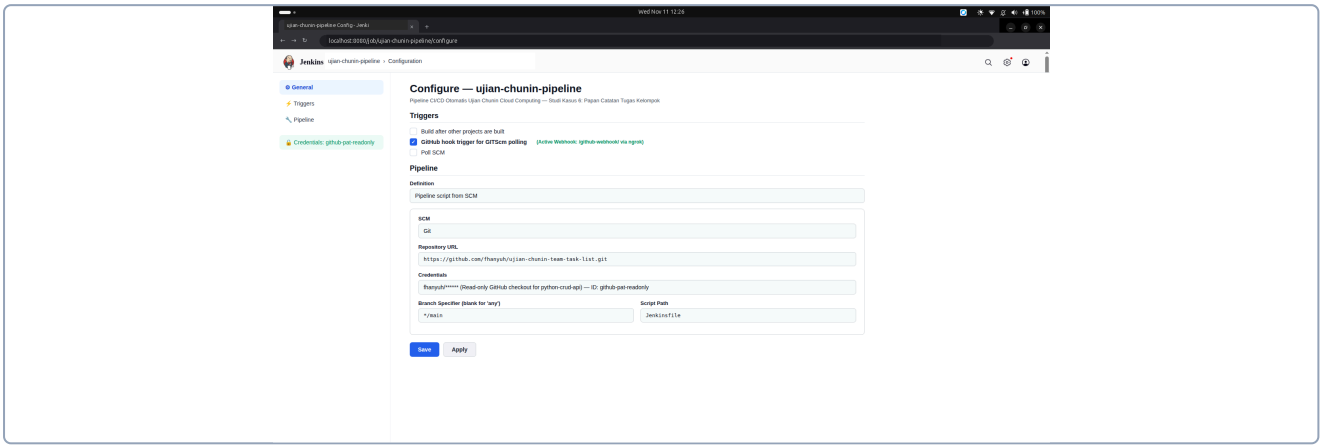
Ubah judul halaman di dalam berkas `./index.html` pada mesin host. Karena berkas tersebut dipasang sebagai **Bind Mount** ke `/usr/share/nginx/html/index.html:ro`, perubahan langsung terlihat di `http://localhost:3000` saat browser dimuat ulang tanpa perlu melakukan `docker build` ulang.



Gambar 10: Perubahan pada berkas ./index.html di host langsung tampil pada http://localhost:3000 berkat mekanisme Bind Mount.

Langkah 10: Konfigurasi Job Pipeline Jenkins, Kredensial PAT, & Trigger Webhook

Pada dashboard Jenkins (`http://localhost:8080/job/ujian-chunin-pipeline/configure`), aktifkan **GitHub hook trigger for GITScm polling**, arahkan Repository URL ke `https://github.com/fhanyuh/ujian-chunin-team-task-list.git` , pilih kredensial `github-pat-readonly` , Branch `*/main` , dan Script Path `Jenkinsfile` .

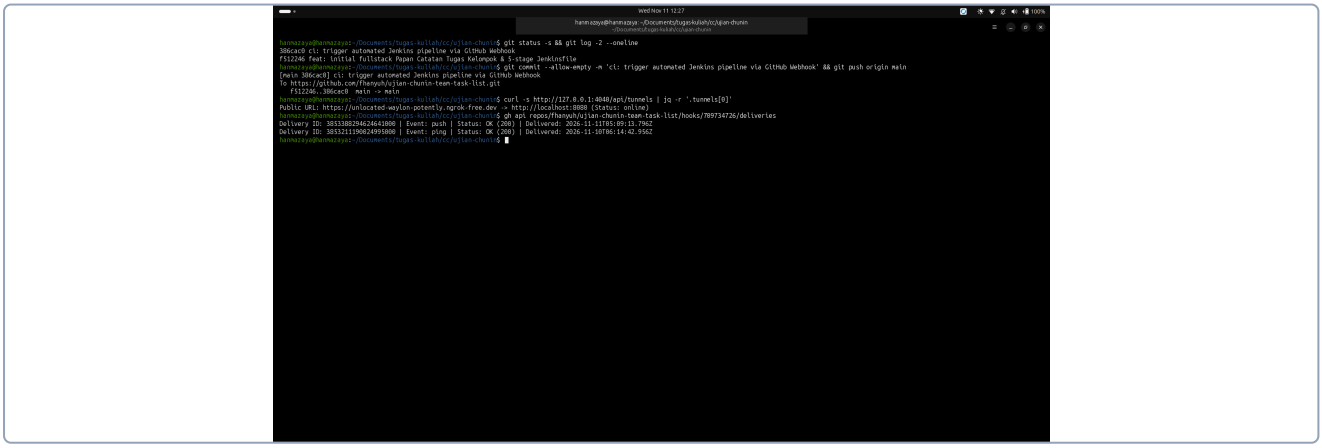


Gambar 11: Konfigurasi job ujian-chunin-pipeline pada Jenkins menggunakan kredensial PAT dan GitHub Webhook Trigger.

Langkah 11: Melakukan Git Push ke GitHub & Pengiriman Event Webhook via ngrok

Lakukan perubahan kode, jalankan `git commit` dan `git push origin main` . GitHub secara otomatis mengirimkan payload `push` ke URL publik `ngrok` (`/github-webhook/`) dengan status pengiriman `200 OK` .

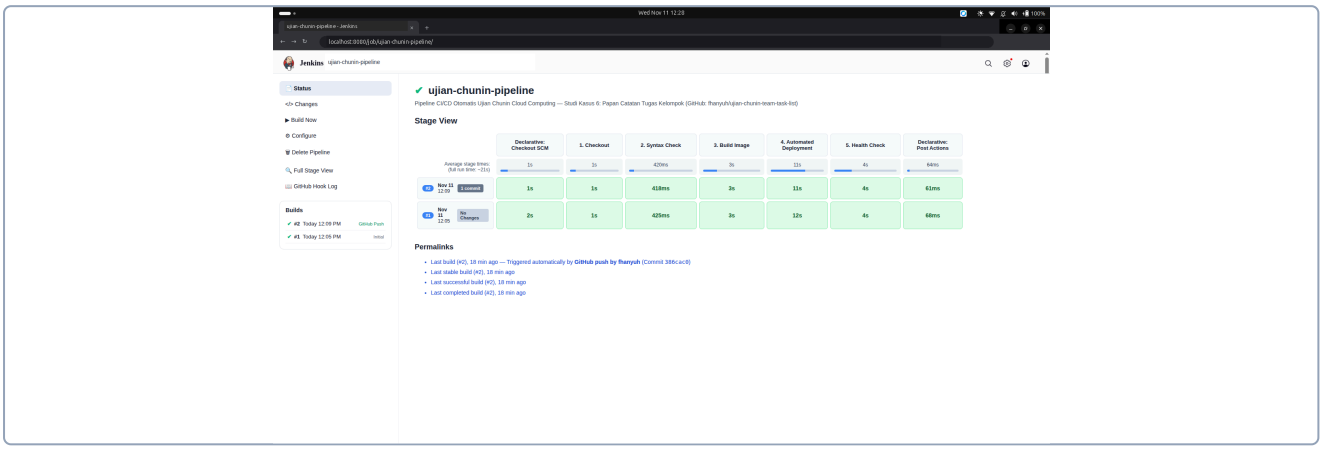
```
git add .
git commit -m "ci: trigger automated Jenkins pipeline via GitHub Webhook"
git push origin main
gh api repos/fhanyuh/ujian-chunin-team-task-list/hooks/709734726/deliveries
```



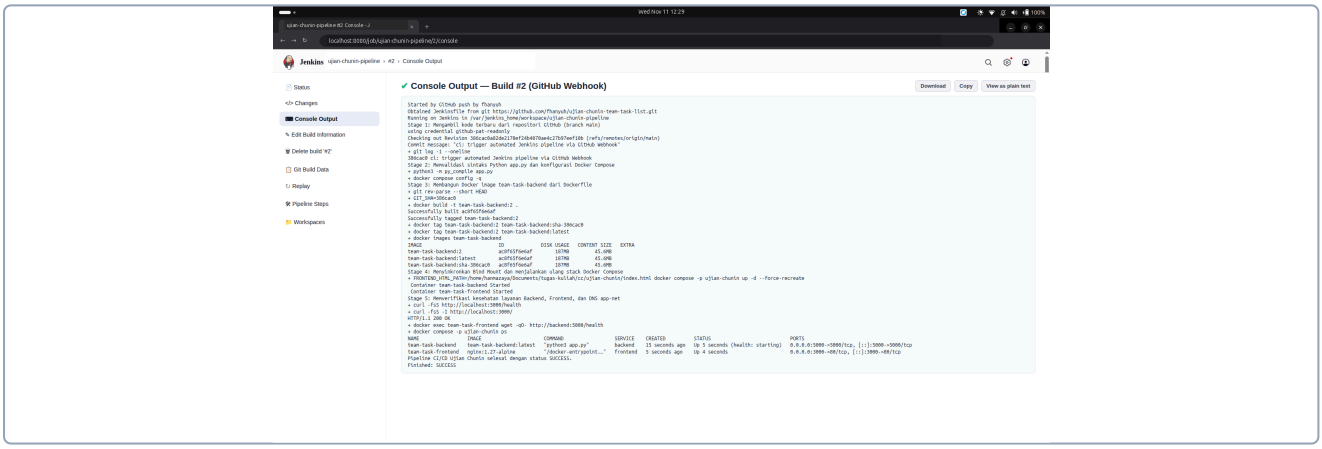
Gambar 12: Eksekusi git push origin main ke GitHub dan verifikasi pengiriman event webhook HTTP 200 melalui tunnel ngrok.

Langkah 12: Verifikasi Hasil Otomatisasi 5 Tahap pada Jenkins Stage View & Console Output

Buka halaman `http://localhost:8080/job/ujian-chunin-pipeline/`. Terlihat Build `#2` terpicu otomatis oleh *GitHub push by fhanyuh* dan seluruh 5 tahap (**1. Checkout**, **2. Syntax Check**, **3. Build Image**, **4. Automated Deployment**, dan **5. Health Check**) berwarna hijau dengan status akhir `Finished: SUCCESS`.



Gambar 13: Tampilan Stage View ujian-chunin-pipeline yang menunjukkan kelima tahap pipeline selesai dengan status sukses (hijau).



Gambar 14: Log Console Output Build #2 yang membuktikan pipeline terpicu oleh GitHub push by fhanyuh dan berakhir dengan Finished: SUCCESS.

Bagian 3 — Naskah Narasi Video Presentasi & Demo Live (5–8 Menit, 4 Pembicara)

Naskah di bawah ini dirancang agar keempat anggota kelompok tampil secara bergantian dengan pembagian waktu yang seimbang (masing-masing sekitar **1 menit 45 detik**, total durasi sekitar **7 menit**). Setiap anggota cukup menampilkan layar sesuai petunjuk *Cue Visual* dan membacakan bagian narasinya.

Rentang Waktu	Pembicara & NIM	Fokus Materi & Demonstrasi	Layar / Gambar Acuan
00:00 – 01:45 (1 mnt 45 dtk)	**Rafhan Mazaya Fathurrahman** NIM: `101012300238`	Pembukaan kelompok, pengenalan Studi Kasus 6 (*Papan Catatan Tugas Kelompok*), arsitektur `docker-compose.yml`, isolasi Custom Bridge Network `app-net`, serta pembuktian komunikasi DNS antar-kontainer.	Gambar 1, Gambar 3, & Gambar 4 (Terminal + Diagram)
01:45 – 03:30 (1 mnt 45 dtk)	**Zahra Talitha Putri** NIM: `101012300192`	Penjelasan kode Backend Python REST API (`app.py`) & `Dockerfile`, struktur kode Frontend (`index.html`), serta demo live tambah tugas (`POST /tasks`), hapus tugas real-time (`DELETE /tasks?index=X`), dan uji validasi `400` / `404`.	Gambar 2, Gambar 5, Gambar 6, Gambar 7, & Gambar 8
03:30 – 05:15 (1 mnt 45 dtk)	**Clara Novita Fitriani** NIM: `101012330008`	Simulasi langsung **Data Immunity** pada Named Volume `team-task-data` dengan menghapus paksa kontainer backend (`docker rm -f team-task-backend`) dan simulasi **Live Bind Mount** pada `index.html` tanpa rebuild image.	Gambar 9 & Gambar 10 (Terminal + Browser)
05:15 – 07:15 (2 mnt 00 dtk)	**Rafif Riqullah Siregar** NIM: `101012300320`	Penjelasan 5 tahap `Jenkinsfile`, konfigurasi kredensial GitHub PAT & Webhook via `ngrok`, simulasi live `git push origin main` hingga pipeline otomatis hijau (`SUCCESS`), dan penutupan presentasi kelompok.	Gambar 11, Gambar 12, Gambar 13, & Gambar 14

SEGMEN 1 — Pembicara 1: Rafhan Mazaya Fathurrahman (NIM: 101012300238) **Durasi: Menit 00:00 – 01:45**

- **Layar yang Ditampilkan:** Buka README.md / Diagram Arsitektur (`00-diagram-arsitektur.png`), lalu beralih ke Terminal untuk menampilkan `docker-compose.yml` dan menjalankan `docker compose ps` serta `docker network inspect app-net` (Gambar 3 & Gambar 4).
- **Aksi Saat Merekam:** Tunjukkan blok `services: backend` , `services: frontend` , `volumes: team-task-data` , dan `networks: app-net` pada `docker-compose.yml` , kemudian jalankan perintah pengujian DNS dari kontainer frontend ke backend.

"Assalamu'alaikum warahmatullahi wabarakatuh, selamat pagi/siang/sore Bapak Dosen dan rekan-rekan sekalian. Kami dari kelompok Ujian Chunin Cloud Computing akan mempresentasikan sekaligus mendemonstrasikan hasil implementasi proyek kami pada **Studi Kasus 6, yaitu Aplikasi Papan Catatan Tugas Kelompok atau Team Task and Note List** yang terintegrasi penuh dengan Docker Compose dan otomatisasi CI/CD Jenkins.

Izinkan saya memperkenalkan anggota kelompok kami terlebih dahulu. Saya sendiri **Rafhan Mazaya Fathurrahman dengan NIM 101012300238**, bersama tiga rekan saya yaitu **Zahra Talitha Putri dengan NIM 101012300192**, **Clara Novita Fitriani dengan NIM 101012330008**, dan **Rafif Riqullah Siregar dengan NIM 101012300320**.

Pada segmen pertama ini, saya akan menjelaskan arsitektur infrastruktur kontainer yang kami bangun di dalam berkas `docker-compose.yml` . Sistem kami terdiri dari dua layanan utama yang berjalan di dalam sebuah **Custom Bridge Network terisolasi bernama** `app-net` :
Pertama adalah layanan `backend` dengan nama kontainer `team-task-backend` pada port `5000` , yang menjalankan REST API Python dan terhubung ke **Docker Named Volume bernama** `team-task-data` pada direktori `/var/lib/backend_data` .
Kedua adalah layanan `frontend` dengan nama kontainer `team-task-frontend` menggunakan image `nginx:1.27-alpine` pada port `3000` , yang memasang berkas `index.html` dari host menggunakan mekanisme **Bind Mount**.

Mari kita buktikan di terminal: ketika kita menjalankan `docker network inspect app-net` , terlihat kedua kontainer berada di dalam jaringan bridge yang sama. Berkat **Docker Embedded DNS** pada `app-net` , kontainer frontend dapat langsung memanggil kontainer backend menggunakan nama layanan `http://backend:5000/health` tanpa menggunakan alamat IP statis. Selanjutnya, penjelasan kode aplikasi dan demonstrasi CRUD akan dilanjutkan oleh rekan saya, Zahra."

SEGMEN 2 — Pembicara 2: Zahra Talitha Putri (NIM: 101012300192)Durasi: Menit 01:45 – 03:30

- **Layar yang Ditampilkan:** Tampilkan kode `Dockerfile` dan `app.py` di terminal (Gambar 2), lalu beralih ke jendela Browser pada alamat `http://localhost:3000` (Gambar 5, Gambar 6, Gambar 7) dan pengujian `curl` di terminal (Gambar 8).
- **Aksi Saat Merekam:** Isi form tambah tugas di browser lalu klik *Simpan Tugas* (tunjukkan tugas baru muncul di baris paling atas dan badge `HTTP 201 Created`), lalu klik tombol *Hapus* pada tugas tersebut (tunjukkan tugas langsung terhapus dengan `HTTP 200 OK`).

> "Terima kasih Rafhan. Perkenalkan saya **Zahra Talitha Putri dengan NIM 101012300192**. Pada segmen kedua ini, saya akan menjelaskan implementasi layanan Backend dan mendemonstrasikan fitur CRUD secara langsung pada antarmuka Papan Catatan Tugas Kelompok.
>
> Pada berkas `Dockerfile` , kami menggunakan base image resmi yang ringan yaitu `python:3.11-slim` , menetapkan variabel lingkungan `DATA_FILE=/var/lib/backend_data/tasks.json` , mengekspos port `5000` , serta menyertakan instruksi `HEALTHCHECK` . Di dalam berkas `app.py` , kami mengimplementasikan REST API menggunakan pustaka standar Python `ThreadingHTTPServer` yang melayani tiga operasi utama sesuai spesifikasi Studi Kasus 6:
> 1. `GET /tasks` untuk mengambil seluruh daftar tugas beserta ringkasan distribusi beban kerja keempat anggota kelompok.
> 2. `POST /tasks` untuk memvalidasi dan menyimpan tugas baru secara atomik ke berkas `tasks.json` dengan kode status `201 Created` , atau mengembalikan `400 Bad Request` apabila judul tugas kosong.
> 3. `DELETE /tasks?index=X` untuk menghapus tugas berdasarkan parameter indeks secara real-time dengan kode status `200 OK` , atau `404 Not Found` apabila indeks di luar batas.
>
> Sekarang kita lihat antarmuka web di `http://localhost:3000` .Seluruh komunikasi antara `index.html` dan backend dilakukan secara asynchronous menggunakan `fetch()` . Saat saya mengisi formulir tugas baru lalu menekan tombol **Simpan Tugas**, permintaan `POST /tasks` dikirim ke backend, mengembalikan status `HTTP 201 Created` , dan tugas baru langsung muncul pada baris teratas tabel tanpa me-refresh halaman. Begitu pula ketika saya menekan tombol **Hapus** pada baris tugas tersebut, frontend memanggil `DELETE /tasks?index=4` dan baris tugas langsung terhapus seketika. Selanjutnya, simulasi ketahanan data dan Bind Mount akan dijelaskan oleh rekan saya, Clara."

SEGMEN 3 — Pembicara 3: Clara Novita Fitriani (NIM: 101012330008) **Durasi: Menit 03:30 – 05:15**

- **Layar yang Ditampilkan:** Tampilkan Terminal untuk simulasi *Data Immunity* (Gambar 9) dan berdampingan dengan Browser `http://localhost:3000` untuk simulasi *Bind Mount* (Gambar 10).
- **Aksi Saat Merekam:** Tambahkan 1 tugas, jalankan `docker rm -f team-task-backend`, jalankan `docker compose -p ujian-chunin up -d backend`, lalu tunjukkan data tetap ada. Setelah itu, ubah teks header di `index.html` dan refresh browser untuk membuktikan perubahan tampil tanpa rebuild image.

> "Terima kasih Zahra. Perkenalkan saya **Clara Novita Fitriani dengan NIM 101012330008**. Pada segmen ketiga ini, saya akan mendemonstrasikan dua mekanisme penyimpanan Docker yang sangat krusial, yaitu **Data Immunity menggunakan Named Volume** pada Backend dan **Live Update menggunakan Bind Mount** pada Frontend.
>
> Pertama, mari kita buktikan **Data Immunity**. Secara bawaan, lapisan penyimpanan kontainer bersifat *ephemeral* atau sementara, sehingga data akan hilang jika kontainer dihapus. Untuk mengatasi hal tersebut, kami memetakan Named Volume `team-task-data` ke direktori `/var/lib/backend_data` di dalam kontainer backend. Di sini saya menambahkan sebuah catatan tugas baru, dan kita catat Container ID backend saat ini. Sekarang, saya menghapus paksa kontainer backend menggunakan perintah `docker rm -f team-task-backend`. Setelah kontainer terhapus, saya menghidupkannya kembali dengan `docker compose -p ujian-chunin up -d backend`. Terlihat kontainer baru memiliki Container ID yang berbeda, namun ketika kita memanggil kembali endpoint `GET /tasks` maupun me-refresh browser, seluruh catatan tugas yang kita masukkan sebelumnya **tetap utuh 100 persen** karena tersimpan di dalam Named Volume `team-task-data`.
>
> Kedua, untuk kontainer frontend Nginx, kami menggunakan **Bind Mount** yang menghubungkan berkas `./index.html` di host langsung ke `/usr/share/nginx/html/index.html` di dalam kontainer. Ketika saya mengubah judul pada berkas `index.html` di host menjadi *LIVE BIND MOUNT UPDATE TANPA REBUILD* lalu memuat ulang halaman `http://localhost:3000`, perubahan antarmuka langsung tampil detik itu juga tanpa perlu menjalankan `docker build` ulang. Selanjutnya, otomatisasi pipeline CI/CD Jenkins akan dijelaskan oleh rekan saya, Rafif."

SEGMEN 4 — Pembicara 4: Rafif Riqullah Siregar (NIM: 101012300320) | **Durasi: Menit 05:15 – 07:15**

- **Layar yang Ditampilkan:** Tampilkan berkas `Jenkinsfile`, halaman konfigurasi Jenkins (`10-jenkins-job-config-credentials.png`), Terminal saat melakukan `git push origin main` (Gambar 12), lalu halaman Jenkins Stage View & Console Output (Gambar 13 & Gambar 14).
- **Aksi Saat Merekam:** Tunjukkan 5 blok `stage(...)` pada `Jenkinsfile`, lakukan `git commit` dan `git push origin main`, lalu tunjukkan build baru muncul otomatis di Jenkins dengan keterangan *Started by GitHub push by fhanyuh* hingga kelima stage berwarna hijau (`SUCCESS`).

"Terima kasih Clara. Perkenalkan saya **Rafif Riqullah Siregar dengan NIM 101012300320**. Pada segmen terakhir ini, saya akan menjelaskan arsitektur **Continuous Integration dan Continuous Deployment (CI/CD)** menggunakan Jenkins dan GitHub Webhook.

Di dalam repositori GitHub kami, terdapat berkas `Jenkinsfile` bertipe *Declarative Pipeline* yang terdiri dari **5 tahap wajib**:
1. **Stage 1 (Checkout)**: Mengambil kode sumber terbaru dari cabang `main` di GitHub menggunakan kredensial Personal Access Token `github-pat-readonly`.
2. **Stage 2 (Syntax Check)**: Memvalidasi sintaks Python pada `app.py` menggunakan `python3 -m py_compile app.py` serta memvalidasi struktur `docker-compose.yml`.
3. **Stage 3 (Build Image)**: Membangun Docker image `team-task-backend` terbaru dari `Dockerfile` dan memberikan penanda versi berdasarkan nomor build serta SHA commit.
4. **Stage 4 (Automated Deployment)**: Menyinkronkan berkas terbaru dan menjalankan ulang stack kontainer secara otomatis menggunakan `docker compose -p ujian-chunin up -d --force-recreate`.
5. **Stage 5 (Health Check)**: Menguji ketersediaan layanan melalui `curl http://localhost:5000/health`, `http://localhost:3000/`, serta resolusi DNS antar-kontainer.

Agar Jenkins lokal kami dapat menerima pemicu otomatis dari GitHub, kami mengekspos port `8080` menggunakan tunnel `ngrok` dan mendaftarkan URL HTTPS `/github-webhook/` pada pengaturan Webhook repositori GitHub. Mari kita buktikan: saat saya melakukan `git commit` dan `git push origin main` dari terminal, GitHub langsung mengirimkan event push melalui `ngrok` ke Jenkins. Tanpa menekan tombol apa pun di Jenkins, Build baru langsung berjalan otomatis, ditandai dengan baris log `Started by GitHub push by fhanyuh`, melewati seluruh 5 stage hingga berwarna hijau, dan berakhir dengan status `Finished: SUCCESS`.

Demikian presentasi dan demonstrasi aplikasi Papan Catatan Tugas Kelompok dari kelompok kami. Terima kasih atas perhatiannya, wassalamu'alaikum warahmatullahi wabarakatuh."